scientific reports

Check for updates

OPEN

A DNA-based color image cryptosystem using chaotic maps, spiral mixing and non-linear binary operator

Chiranjeev Bhaya[1,3], Mohd Zain[2,3] & Abhay Kumar Singh[2,3]✉

The goal of image cryptosystems is to protect image transmission when there are network adversaries present. To ensure secrecy, images are subject to encryption to produce unintelligible cipher images; the techniques used for this process differ significantly from those applied to text data. The majority of the cryptosystems consider complicated or confusion–diffusion architectures that change and permute the values of the pixels. These frequently entail binary operations like bitwise XOR, addition–subtraction, DNA operations, etc, and are carried out utilizing chaotic maps, each having certain limitations. This paper employs a non-linear binary function applicable to traditional and DNA techniques for color image cryptosystems. In the proposed method, each of the color components of the image follows some steps which start with inter-channel mixing and mix rows, taking the values from the initial vector and mixing it. Arnold's cat map algorithm is then applied to shuffle the pixels, and a novel spiral mixing of the pixels is applied in four different directions to prevent the differential attack. Finally, encoding, substitution, and decoding based on DNA is carried out. A multiple collapse chaotic map is used to derive initialization vectors, rule maps, and DNA substitution maps which are used in encoding and decoding processes. Experimental results and analysis show that the proposed cryptosystem results in cipher images that are resistant to different types of cryptographic attacks, thereby providing a secure color image encryption and decryption scheme.

With the advent and ease of availability of modern technologies like digital cameras, smartphones, satellite imaging systems, medical imaging systems, etc., the amount of image data has been increasing exponentially year after year. Not only quantitatively, but this growth is also seen qualitatively, which is resulting in the capture, processing and transmission of such data in high volume. This large amount of data needs effective storage tool, as well as mechanisms to secure the information contained in them. The use of *Deoxyribo Nucleic Acid* (DNA) is proving to be a promising technology to solve this problem[1].

One of the ways to protect data against malicious use is to convert them into unintelligible form by the use of cryptography *encryption algorithms*. These methods use the knowledge of what is known as an "*encryption key*" to convert the data into what is known as the *cipher*, which can be regenerated into its original form only with the knowledge of the correct key using a corresponding *decryption algorithm*[2]. However, unlike text, images are huge in size and are information contained in them are highly correlated, therefore, *symmetric key cryptosystems* with complex *confusion* and *diffusion* algorithms are used to encrypt them[3]. The resultant cipher image, thus formed, looks like a noise pattern and cannot be correctly deciphered without the knowledge of the corresponding symmetric key.

The use of DNA in image cryptosystems enhance the confusion and diffusion processes involved in them. The data is represented in the form of DNA molecules, which is a sequence of the constituent four nitrogenous bases, namely *Adenine*, *Cytosine*, *Guanine*, and *Thiamine*, represented in this paper as A, C, G, and T, respectively. Various DNA encoding and decoding rules are used to convert the binary data into DNA nucleotides and vice-versa[3]. The nucleotides are operated among each other using various DNA rules of addition, subtraction, XOR, etc.[3]. A detail of some of the image encryption algorithms is provided in the following subsection.

[1]Department of Computer Science and Engineering, Indian Institute of Technology (Indian School of Mines), Dhanbad, Jharkhand 826004, India. [2]Department of Mathematics and Computing, Indian Institute of Technology (Indian School of Mines), Dhanbad, Jharkhand 826004, India. [3]Chiranjeev Bhaya, Mohd Zain and Abhay Kumar Singh contributed equally to this work. ✉email: itbhu81@gmail.com

An image is represented as a matrix of rows and columns and are generally huge in size. To apply confusion and diffusion algorithms, some pseudo-random sequences generated from some small seed values are therefore needed. Most of the image cryptosystems, therefore, use *chaotic maps* to produce these sequences, which are obtained from the symmetric key values. Researchers around the globe have developed a number of such maps, like the *2D-Infinite Collapse Map*[4], *Logistic Maps*[5] *3D Chen chaotic sysytem*[4], *Hyper Chaotic Systems*[6], *Logistic Tent System*[7], *4D Memristive Hyper Chaos*[8], *Fractional Order Hyperchaotic System*[9], *Fractional Fourier Transform*[10] etc., which are highly sensitive to the initial parameters obtained from the key and produce sequences with varying size and dimensions.

Since images have a high inter-pixel correlation, standard encryption algorithms which are implemented in binary data encryption prove to be ineffective[3]. Therefore, most of the image based cryptosystems follow a complex architecture, containing a number of steps for confusion and diffusion of image pixel values. For example, Chai et al.[4] diffusion and permutation of image pixels using pseudorandom sequences obtained from *Chen chaotic system*. Moattar and Niyat[6] first decomposed the images into the constituent color channels and later used permutation followed by DNA based encoding and diffusion, and a second round of confusion. Liu et al.[8] used dynamic dispersion and dispersal in the DNA encoded color matrices of the images followed by DNA decoding and component integration. Farah et al.[10] used operations like DNA emcoding, *Fractional Fourier Transform* and XOR in various stages of the encryption algorithm.

A number of algorithms have been developed recently which perform a number of DNA-based operations to encrypt images. Benyahia et al.[11] introduced a hybrid encryption scheme combining methods of DNA computing with Lorenz chaotic dynamics to enhance security. The method encoded the image pixels into DNA sequences and then scrambled them using chaotic maps. The cryptosystem had a large key space, high-sensitivity to key variants, and a strong resistance to differential and statistical attacks. However, it had a very high computational complexity, making it difficult for real time implementation. Fetteha et al.[12] designed an encryption algorithm for grayscale images using DNA coding and chaotic systems which was optimized for *Field Programmable Gate Array* (FPGA). The algorithm was designed in such a way that it would use minimum hardware resources with a strong resistance to differential attacks. However, it was primarily designed for grayscale images, limiting its applicability to color images. Dieu et al.[13] formulated a new chaotic system without linear terms, and its applications on DNA-based image cryptosystems. The proposed algorithm had high entropy values, strong resistance to brute-force and differential attacks. Alrubaie et al.[14] proposed a double dynamic DNA sequence encryption algorithm using a 2D logistic map. The method gave a strong encryption with a high resistance to statistical attacks, but required a precise key management to avoid vulnerabilities. Kheiri and Dehghani[15] explored the integration of DNA computing with cellular automata and chaos theory for image encryption. The method enhanced the randomness and security of the cryptosystem using cellular automata, but required a careful parameter selection to maintain the encryption strength.

Most of the color image cryptosystems use operations like XOR, addition-subtraction $(+, -)$, which have their own limitations, which was illustrated in Mishra et al.[2]. To overcome their limitations, they devised a *novel non-linear binary operator* and used it in a proposed grayscale and medical image cryptosystem. In this paper, we leverage the properties of this operator $(\otimes)$ to design a novel color-image cryptosystem. The algorithm implements novel methods like *inter-channel* and *spiral mixing*, and generalized methods like modified *Arnold's Cat Map* (ACM) to design a robust encryption scheme. The proposed algorithm encrypts color images of varying dimensions which results in a cipher that looks noise like and can resist various cryptography attacks. The cipher can be losslessly decrypted into the original image using the correct encryption key.

The major contributions of this paper are as follows.

- The proposed algorithm uses novel methods like *inter-channel* and *spiral* mixing, along with traditional methods like *mix rows* with the binary operator $\otimes$ to reduce the visual information and increase the entropy of the original image.
- The proposed cryptosystem uses a number of confusion and diffusion processes to generate noise-like cipher images using techniques like *modified ACM* for confusion and pseudorandom values from *2D-Multiple Collapse map*[16] for the diffusion steps.
- The proposed cryptosystem has a non-linear mapping from the original image and the generated cipher image by the use of the non-linear binary operator $\otimes$ and use of operations like DNA encoding, substitution and DNA decoding in its various steps.
- The security of the proposed cryptosystem has been tested against a number of statistical and security parameters, and the experimental results show its resistance against various cryptography attacks.
- The comparison analysis of the non-linear binary operator $\otimes$ with conventional operators such as XOR and $(+, -)$ show that the former performs better than them, and therefore can be used for efficient and secure color image cryptosystems.

The following section describes the proposed method in details.

Methods

The algorithm for the proposed encryption and decryption methods are illustrated in Figs. 5 and 8 respectively. The proposed algorithm uses a *2D-Multiple Collapse Chaotic Map* as defined in Ref.[16], the initial parameters of which are obtained by the symmetric 240-bit key using the *key scheduling* algorithm. This chaotic map is used to generate various pseudo-random sequences for initial vector, rule maps and DNA substitution maps, which are used for the various encryption steps. Similar maps are used in the decryption algorithm for the respective reverse steps to generate back the original image using the correct key that was used for encryption. The detailed steps of the algorithm are explained in the following subsections.

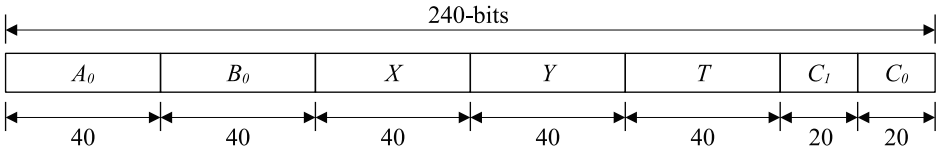


Fig. 1. Deriving the initial parameters of the chaotic map using a 240-bit symmetric key.

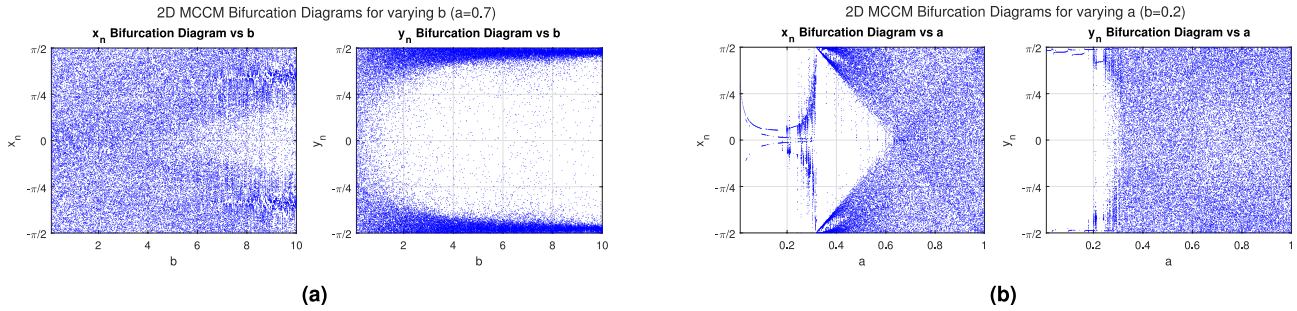


Fig. 2. Bifurcation diagram of 2D-MCMM chaotic map.

Key scheduling

The proposed cryptosystem works on a symmetric key of 240 bits, which is used for both encryption as well as decryption. The key is used to obtain the initial values of the chaotic map, based on which the other chaotic parameters are to be determined. The determination of the initial values from the 240-bit long key is shown in Fig. 1. The key is broken into seven parts, first five of which are 40 bit long, and the last two are of 20 bits each. The values, A_0, B_0, X, Y, T, C_1, and C_0, thus respectively obtained, are used to calculate the initial parameters of the chaotic maps $\langle x_0, y_0, a, b\rangle$ using the equations

$$x_0 = (X + T \times C_1) \bmod 2 - 1,$$
$$y_0 = (Y + T \times C_0) \bmod 2 - 1,$$
$$a = (A_0 + T \times C_1) \bmod 5 + 16,$$
$$b = (B_0 + T \times C_0) \bmod 5 + 16,$$

where $(a) \bmod m$ is defined as the sum of the integral remainder when a is divided by m and the fractional part of a.

Chaotic map generation

The proposed algorithm uses 2D-MCMM to generate the pseudorandom sequence used in the encryption and decryption algorithm. This map generates chaotic sequences using the equations defined in Yang et al.[16] as

$$x_{n+1} = \tan^{-1}\left(\frac{b}{10 \times a \times y_n} + \tan\left(a \times \pi \times x_n\right)\right),$$

and

$$y_{n+1} = \tan^{-1}\left(\frac{b}{a \times x_n} + \tan\left(10 \times a \times \pi \times y_n\right)\right),$$

where a and b, are the tuning parameters derived from the symmetric key.

The chaotic nature of the 2D-MCMM can be seen in its bifurcation diagram which is presented in Fig. 2. The diagram shows a high chaotic nature of the map, which makes it suitable for an image cryptosystem. The phase diagram, as shown in Fig. 3, demonstrates a periodic motion of the chaotic map, depicting that the trajectory is periodic and not confined to a specific bounded region. Similarly, the Lyapunov exponents of the chaotic map shown in Fig. 4 demonstrate the sensitivity of the chaotic parameters to its initial values. A detailed analysis of the chaotic properties of the map, along with comparisons with other chaotic maps can be found in Yang et al.[16].

The generation of the map starts with some initial values (x_0, y_0) which are also derived from the key and the subsequent sequences are obtained by iteratively using the above equations. The proposed algorithm requires three different types of pseudorandom sequences, the detailed of which are explained in the subsections below.

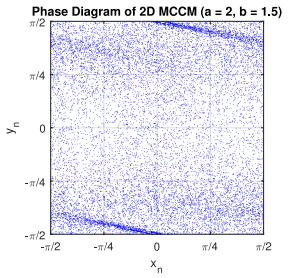


Fig. 3. Phase diagram of 2D-MCMM chaotic map.

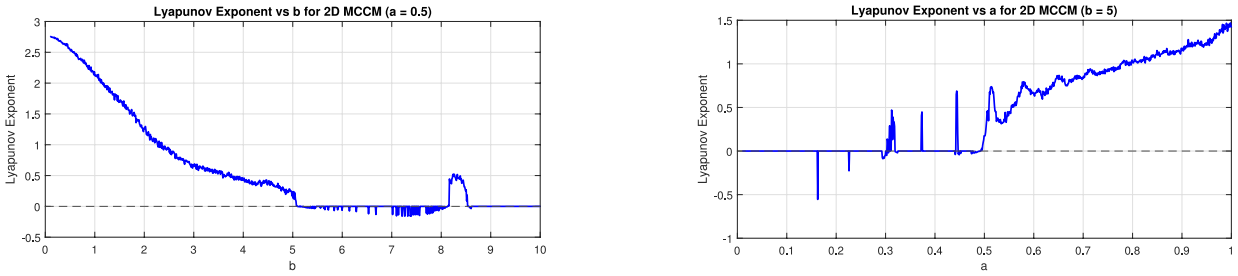


Fig. 4. Lyapunov exponent of 2D-MCMM chaotic map.

Binary value	Rule 1	Rule 2	Rule 3	Rule 4	Rule 5	Rule 6	Rule 7	Rule 8
00	A	C	A	G	C	T	G	T
01	C	A	G	A	T	C	T	G
10	G	T	C	T	A	G	A	C
11	T	G	T	C	G	A	C	A

Table 1. DNA encoding rules.

Initial vector

Am initial vector is used to mix the rows of each channel of the color image so as to increase its entropy. For an image of dimension $m \times n$, the initial vector has a dimension of $1 \times m$ for each channel, the values of which are obtained as $IV(1,i) = (|x_i| \times |y_i| \times (2^{31}-1)) \bmod (2^p))$, where $i \in [1,m]$, x_i and y_i are obtained from the 2D-MCCM, and p is the pixel-depth of the image. Natural images generally have a pixel depth of 8, whereas some medical images have a pixel depth of 16. All the images that have been used in the experimental analysis of this paper have a pixel depth of 8.

Rule map

In the DNA encoding and decoding processing steps of the algorithm, one of the eight valid rules are to be applied for each pixel operation, for which the sequences are obtained from the *Rule Map*. Since every two nucleotides are mapped to one pixel, the dimension of the rule map for an image of size $m \times n$, the dimension of the rule maps for each channel would be $m \times (n \cdot p/2)$, where p is the pixel depth. To generate the rule map, the chaotic sequence obtained from the 2D-MCCM chaotic map is converted to integer values in the range $[0, 2^{3p/2}]$ of dimension $m \times n$ an then $p/2$ rules are obtained from every element and scaled to a range of [1, 8]. This obtained value is used to select the DNA encoding and decoding rules as defined in Tables 1 and 2 respectively.

DNA substitution map

The non-linear binary operator $\otimes$ is used for DNA substitution. In this process, the DNA nucleotides are mapped to integer values as $f : \{A, C, G, T\} \rightarrow \{0, 1, 2, 3\}$ using a one-to-one mapping. The substitution operation for two DNA nucleotides with integer values x_1 and x_2 are evaluated based on the definition of $\otimes$ defined in Ref.[2] as $x_1 \otimes x_2 = ((x_1 + 1) \times (x_2 + 1) \bmod 5) - 1$ The value obtained is then mapped back to the DNA nucleotide value using the same reverse mapping. Once the pseudorandom sequences are obtained, the images can be encrypted using the proposed encryption algorithm as described in the following section.

Nucleotide	Rule 1	Rule 2	Rule 3	Rule 4	Rule 5	Rule 6	Rule 7	Rule 8
A	00	01	00	01	10	11	10	11
C	01	00	10	11	00	01	11	10
G	10	11	01	00	11	10	00	01
T	11	10	11	10	01	00	01	00

Table 2. DNA decoding rules.

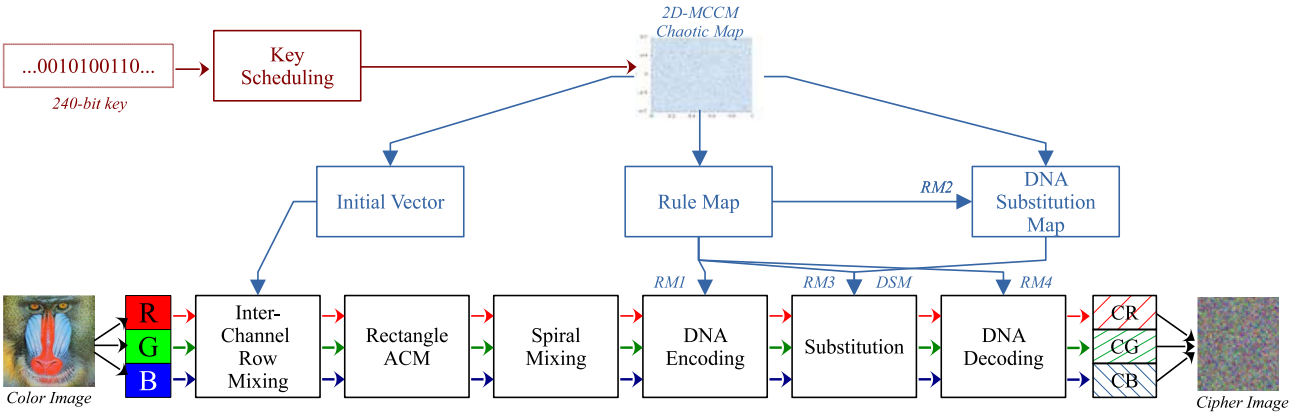


Fig. 5. Proposed encryption algorithm.

Encryption algorithm

The proposed encryption algorithm is shown in Fig. 5. This paper uses the operator $\otimes$ defined in Mishra et al.[2] as $\alpha \otimes \beta = -1 + \{(1+\alpha) \times (1+\beta)\} \bmod p$ for some prime p and $\alpha, \beta \in \mathbb{Z}_{p-1}$. The first stage involves intermixing channels of the image and then utilizing the proposed operator $\otimes$ to operate the rows of the intermixed images with the initial array. Using a modified version of Arnold's Cat Map[17], which can be applied to both square and rectangular images, the pixels of the composite image are jumbled. The suggested operator is then used to further mix the pixels in the directions of forward spiral row, forward spiral column, reverse spiral row, and reverse spiral column. These keyless procedures are necessary to eliminate the link between pixels and the image's visual perception. Lastly, a non-linear layer has been added by combining the suggested operator with the idea of DNA cryptography. One of the eight encoding principles identified by the rule chart is applied to the mixed image that was obtained in the preceding phases, converting it into a sequence of DNA nucleotides. At this stage DNA encoded image is obtained and these encodings are further substituted using the proposed operator with the substitution map. Lastly, one of the eight DNA decoding guidelines identified by a different rule chart is applied to decode the substituted DNA sequences.

Each step of the encryption algorithm is explained in the following subsections.

Splitting of the color channels

Split the R, G, and B pixel matrices from the color image into three different matrix say red, green, blue. Once the channels are split, we need to apply all steps to every matrix in order to encrypt the image in best way which are described in the following subsections.

Inter channel mixing and mix rows

In this stage, the channels of the image are intermixed with each other using some equations. The values from the initial vector (*IV*), which has already been generated, are mixed and distributed throughout the inter mixed image's rows. The IV will have a dimension of $1 \times m$ for a plain image I with dimensions $m \times n$. To create image I', the first value in each row of the matrix is operated with $\otimes$ using the appropriate values of the initial array. The channel are Inter Mixed using the equations

$$\begin{aligned} \mathrm{R}' &= \mathrm{R} \otimes (\mathrm{G} \otimes \mathrm{B}), \\ \mathrm{G}' &= \mathrm{G} \otimes (\mathrm{R}' \otimes \mathrm{B}), \\ \mathrm{B}' &= \mathrm{B} \otimes (\mathrm{R}' \otimes \mathrm{G}'). \end{aligned} \tag{1}$$

The value of the first element in each row is propagated along its neighbor pixels to skew the value in the neighboring pixels using the equation $I'(g,h) = I(g,h) \otimes I'(g,h-1); h = 2,3,\ldots,n; \forall g \in 1,\ldots,m$.

The process of inter channel mix rows will be applied on the image received after splitting color image into matrices i.e. R, G, B, respectively as

$$\begin{aligned} \mathrm{R}' &= \texttt{INTER_MIX}(\mathrm{R}, \mathrm{G}, \mathrm{B}), \\ \mathrm{G}' &= \texttt{INTER_MIX}(\mathrm{G}, \mathrm{R}', \mathrm{B}), \\ \mathrm{B}' &= \texttt{INTER_MIX}(\mathrm{B}, \mathrm{R}', \mathrm{G}', \end{aligned} \tag{2}$$

where INTER_MIX() is defined in Algorithm 1.

INTER_MIX(X_1, X_2, X_3):
1 $m, n = \texttt{size}(X_1)$
2 Initialize a double-dimension array IMix of size $m \times n$
3 **for** $i = 1$ **to** m **do**
4 **for** $j = 1$ **to** n **do**
5 $\texttt{IMix}[i][j] = X_1[i][j] \otimes (X_2[i][j] \otimes X_3[i][j])$
6 **return** IMix

Algorithm 1. Channel intermix.

The image channels are then further mixed using the MIX_ROWS algorithm, defined in Algorithm 2, as

$$\begin{aligned} \texttt{mixedIm}_\mathrm{R} &= \texttt{MIX_ROWS}(\mathrm{R}', IV), \\ \texttt{mixedIm}_\mathrm{G} &= \texttt{MIX_ROWS}(\mathrm{G}', IV), \\ \texttt{mixedIm}_\mathrm{B} &= \texttt{MIX_ROWS}(\mathrm{B}', IV). \end{aligned} \tag{3}$$

MIX_ROWS(X, IV):
1 $m, n = \texttt{size}(X)$
2 Initialize a double-dimension array X' of size $m \times n$
3 **for** $i = 1$ **to** m **do**
4 $X'[i][1] = X[i][1] \otimes IV[1][i]$
5 **for** $j = 2$ **to** n **do**
6 $X'[i][j] = X[i][j] \otimes X[i][j-1]$
7 **return** X'

Algorithm 2. Mix rows.

Rectangular ACM

Arnold's Cat Map (ACM)[17] is a chaotic map that is used to rearrange the given image. When a 2D array has dimensions of $L \times L$, ACM is expressed as

$$\begin{bmatrix} g' \\ h' \end{bmatrix} = \begin{bmatrix} 2 & 1 \\ 1 & 1 \end{bmatrix} \times \begin{bmatrix} g \\ h \end{bmatrix} \bmod L,$$

where $g, h, g', h' \in [1, N]$, (g, h) and (g', h') represents the original and scrambled images' position indices respectively. The drawback of employing directly ACM algorithm is that its definition is limited to identical size images. Nonetheless, uneven proportions are a regular occurrence in both natural and medical imaging. The image is separated into minimal squares, and ACM is applied independently to each of them in order to fix this problem.

Let $m \times n$ represent an image's dimension. To divide it into as few squares as feasible, the length of each square's side will be $N = \min(m, n)$. Therefore, the proportion of squares that are obtained will be $\alpha = \lceil \max(m, n)/N \rceil$. If m and n are distinct combinations of one another, then $L = N - (\max(m, n) \bmod N)$, an additional length L pixel remains with the longer side. There will be some overlap between the squares in order to accommodate

this additional space within the image. There can be a maximum of $\alpha - 1$ overlaps to ensure almost equal overlapping; the duration of the last overlap i.e. $\eta = \lfloor L/(\alpha - 1) \rfloor$. The last overlap modifies the excess portion that is still present.

Given that $m < n$ for an image, $N = m$ and the initial value of $I[1][1]$ of the first square, $a = 1, b = 1$. For the initial $\alpha - 1$ squares, the subsequent values will be ascertained as $b' = b + N - \eta$. To accommodate the extra square that remains, the *y*-coordinate for the last square will be $y - N$. For images when $m > n$, same computations can be performed by transposing the *m* and *n* values. Next, ACM is applied to each separated square individually. On the other hand, ACM is regular; that is, the original image is restored following a definite number of iterations. The dimension of the image *N* determines this regularity. To achieve a well-shuffled image and avoid regularity, ACM will be applied to the image itself *t* number of times, where *t* is the highest prime number less than or equal to (*N*/2), *N* being the size of each image.

The process of applying ACM on images will be applied on the image received after `MIX_ROWS` operation on every matrix i.e. R, G, B, respectively as

$$\begin{aligned} \texttt{imageACM}_{\texttt{R}} &= \texttt{APPLY_ACM}(\texttt{mixedIm}_{\texttt{R}}), \\ \texttt{imageACM}_{\texttt{G}} &= \texttt{APPLY_ACM}(\texttt{mixedIm}_{\texttt{G}}), \\ \texttt{imageACM}_{\texttt{B}} &= \texttt{APPLY_ACM}(\texttt{mixedIm}_{\texttt{B}}). \end{aligned} \tag{4}$$

Spiral mixing

In order to prevent differential attacks on the image, this stage job is to propagate the pixel values information throughout it in spiral manner. This procedure helps remove the image's visual information even if it doesn't require an encryption key. Let *m* and *n* represent the row and column counts of the input image *I*, respectively. The spiral mixing is done in four phases—forward row, forward column, reverse row, and reverse column as shown in Fig. 6. The algorithm for spiral mixing is presented in Algorithm 3.

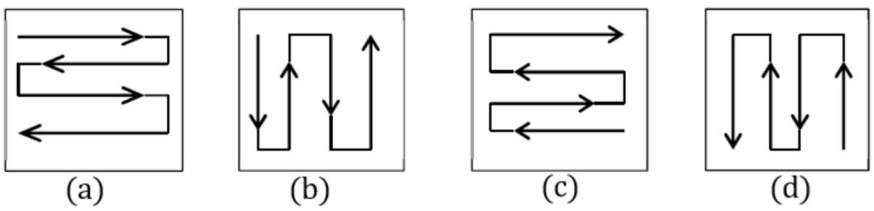


Fig. 6. Spiral mixing—(**a**) Forward Row, (**b**) Forward Column, (**c**) Reverse Row, (**d**) Reverse Column.

```
SPIRAL_MIX(X):
m, n = size(X)
Create an array X' of size m × n
/* -- forward spiral row mixing -- */
for i = 1 to m do
    if i ≅ 1 mod 2 then
        if i > 1 then
            X'[i][1] = X[i][1] ⊗ X'[i − 1][1]
        for j = 2 to n do
            X'[i][j] = X[i][j] ⊗ X'[i][j − 1]
    else
        X'[i][n] = X[i][n] ⊗ X'[i − 1][n]
        for j = n − 1 to 1 do
            X'[i][j] = X[i][j] ⊗ X'[i][j + 1]
/* -forward spiral column mixing - */
for j = 1 to n do
    if j ≅ 1 mod 2 then
        if i > 1 then
            X'[1][j] = X'[1][j] ⊗ X'[1][j − 1]
        for i = 2 to m do
            X'[i][j] = X'[i][j] ⊗ X'[i − 1][j]
    else
        X'[m][j] = X'[m][j] ⊗ X'[m][j − 1]
        for i = m − 1 to 1 do
            X'[i][j] = X'[i][j] ⊗ X'[i + 1][j]
/* -- reverse spiral row mixing -- */
for i = m to 1 do
    if (m − i) ≅ 1 mod 2 then
        X'[i][1] = X'[i][1] ⊗ X'[i + 1][1]
        for j = 2 to n do
            X'[i][j] = X'[i][j] ⊗ X'[i][j − 1]
    else
        if i < m then
            X'[i][n] = X'[i][n] ⊗ X'[i + 1][n]
        for j = n − 1 to 1 do
            X'[i][j] = X'[i][j] ⊗ X'[i][j + 1]
/* -reverse spiral column mixing - */
for j = n to 1 do
    if (n − j) ≅ 1 mod 2 then
        X'[1][j] = X'[1][j] ⊗ X'[1][j + 1]
        for i = 2 to m do
            X'[i][j] = X'[i][j] ⊗ X'[i − 1][j]
    else
        if j < n then
            X'[m][j] = X'[m][j] ⊗ X'[m][j + 1]
        for i = m − 1 to 1 do
            X'[i][j] = X'[i][j] ⊗ X'[i + 1][j]
return X'
```

Algorithm 3. Spiral mixing.

These four steps are required to be processed on the image one after other in continuation on every image taken into consideration. The process of mixing image will be applied on the image received after applying ACM on every matrix i.e. R, G, B, respectively as

$$\begin{aligned} \texttt{imgMix}_{\texttt{R}} &= \texttt{SPIRAL_MIX}(\texttt{imageACM}_{\texttt{R}}), \\ \texttt{imgMix}_{\texttt{G}} &= \texttt{SPIRAL_MIX}(\texttt{imageACM}_{\texttt{G}}), \\ \texttt{imgMix}_{\texttt{B}} &= \texttt{SPIRAL_MIX}(\texttt{imageACM}_{\texttt{B}}). \end{aligned} \tag{5}$$

DNA encoding

One of the encoding principle transforms each pixel value in the image into a string of DNA nucleotides. At a time, two bits are extracted and transformed into a nucleotide by applying the rule found in the obtained rule map shown in Table 1. As a result, a string half of length i.e. $(p/2)$ nucleotides per pixel will be created from an image with a bit depth of p-bit. The rules are obtained by a rule map $RM1$ as shown in Fig. 5. The encoding process of an image will be applied on the image received after mixing on every color matrix as

$$\begin{aligned} \texttt{enc}_{\texttt{R}} &= \texttt{DNA_ENCODE}(\texttt{imgMix}_{\texttt{R}}, RM1), \\ \texttt{enc}_{\texttt{G}} &= \texttt{DNA_ENCODE}(\texttt{imgMix}_{\texttt{G}}, RM1), \\ \texttt{enc}_{\texttt{B}} &= \texttt{DNA_ENCODE}(\texttt{imgMix}_{\texttt{B}}, RM1), \end{aligned} \tag{6}$$

where the `DNA_ENCODE` algorithm is defined in Algorithm 4.

`DNA_ENCODE`(X, RM) **:**

1 $p =$ pixel depth of image X in bits
2 $m, n =$ `size`(X)
3 Initialize a character array `D` of size $m \times n \times (p/2)$
4 **for** i = *1* **to** m **do**
5 $count = 1$
6 **for** j = *1* **to** n **do**
7 `b` = binary value of $X[i][j]$ represented in p-bits
8 **foreach** *two subsequent bits in* `b` **do**
9 `d` = DNA nucleotide for binary values as shown in Table 1 using Rule $RM[i][count]$
10 Append `d` to `D`
11 $count = count + 1$
12 **return** `D`

Algorithm 4. DNA encoding.

Substitution

The DNA sequences D that are received after the image has been encoded are substituted in the replacement process with $\otimes$ using the sequence that is obtained from the DNA replacement Map (DSM). The mapping of $\Sigma = \mathrm{A}, \mathrm{C}, \mathrm{G}, \mathrm{T}$ to $S = 0, 1, 2, 3$ for DNA operation requires the selection of one among eight encoding rules from the values derived from the rule map (RM). Eight options will appear when $\otimes$ is used, one for each of the encoding rules that are applied to a nucleotide pair; yet, some pairs may give identical outcomes for distinct rules. Since of this, the dynamical DNA sequence that is left over after this procedure is resistant to algebraic attack since it depends on the rule map RM as well as the elements of the DNA Substitution Map (DSM), both of which are necessary to decrypt the cryptic image and determine the original key. The algorithm for DNA Substitution is given in Algorithm 5.

DNA_SUBSTITUTION(X, RM, DSM) :
p = pixel depth of image X in bits
m, n = size(X)
Create an array X' of size $m \times n$
for i = *1* **to** m **do**
  for j = *1* **to** n **do**
    x = Numerical value of $X[i][j]$ according to rule $RM[i][j]$ in Table 2
    s = Numerical value of $DSM[i][j]$ according to rule $RM[i][j]$ in Table 2
    $x' = x \otimes s$
    $X'[i][j]$ = DNA value of x' according to rule $RM[i][j]$ in Table 1
return X'

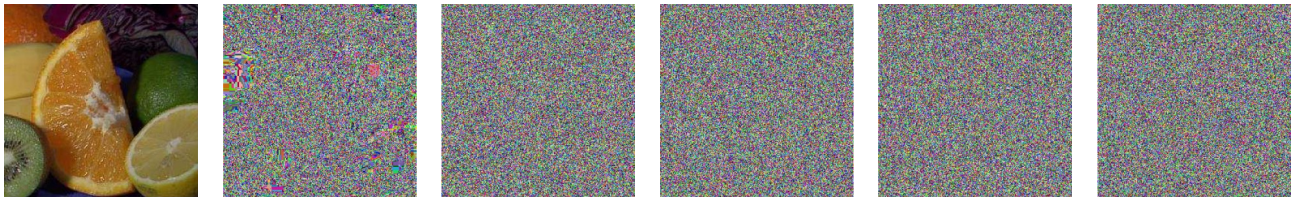
Algorithm 5. DNA substitution.

The rule maps ($RM3$) and the DNA substitution map (DSM) are be used during the substitution procedure as shown in Fig. 5. Following encoding on each matrix, R, G, and B, respectively, the process will be applied to the image received as

$$\begin{aligned} \mathtt{sub_R} &= \mathtt{DNA_SUBSTITUTION}(\mathtt{enc_R}, RM3, DSM), \\ \mathtt{sub_G} &= \mathtt{DNA_SUBSTITUTION}(\mathtt{enc_G}, RM3, DSM), \\ \mathtt{sub_B} &= \mathtt{DNA_SUBSTITUTION}(\mathtt{enc_B}, RM3, DSM). \end{aligned} \tag{7}$$

DNA decoding

One of the eight decoding rules as shown in Table 2 is used to the chaotic DNA sequences in order to transform them to decimal values; the rule map's sequences determine which rule is applied. Then, a single cipher color image is created by combining the three matrices—cipher red, cipher green, and cipher blue obtained after decoding. The outcome of this decoding process is the final cipher image. The algorithm for DNA decoding is presented in Algorithm 6.



(a) Original Image **(b)** After inter channel mixing **(c)** After mix rows **(d)** After rectangular ACM **(e)** After spiral mixing **(f)** After DNA operations (Cipher image)

Fig. 7. Intermediate results of various steps of the proposed encryption algorithm.

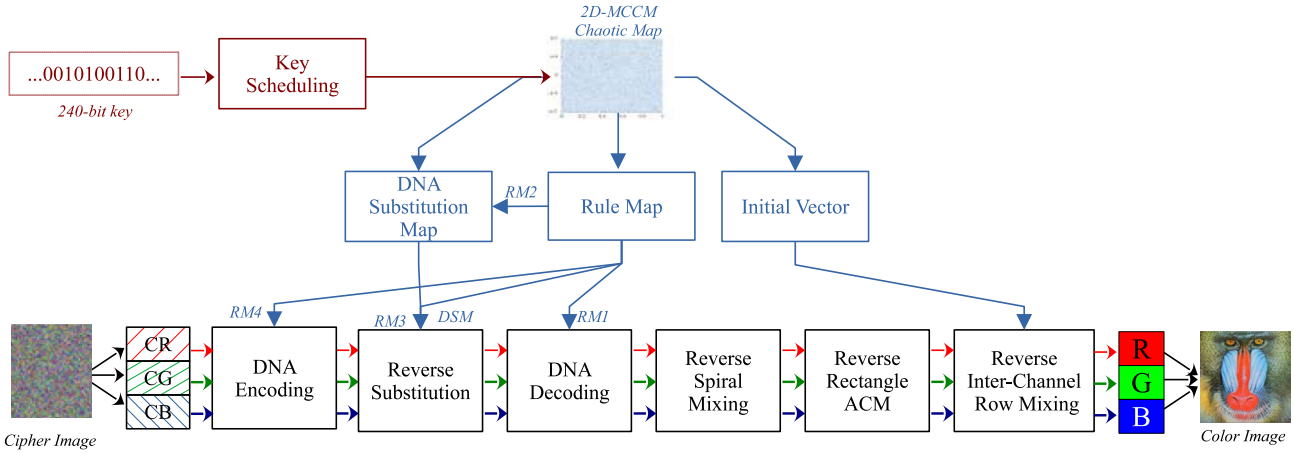
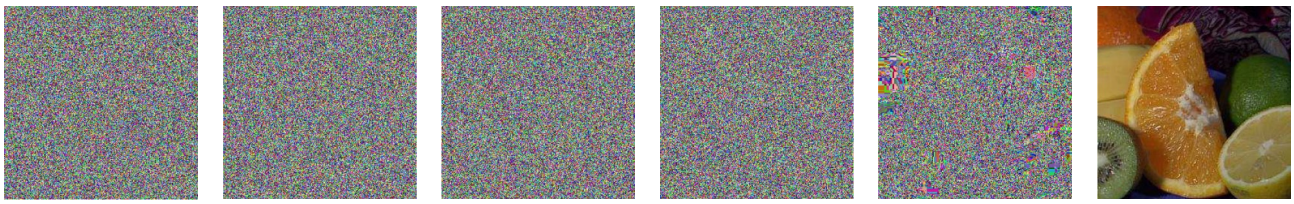


Fig. 8. Proposed decryption algorithm.



(a) Cipher image

(b) After DNA operations

(c) After reverse spiral mixing

(d) After reverse rectangular ACM

(e) After reverse row-mixing

(f) After reverse inter-channel mixing (Original Image)

Fig. 9. Intermediate results of various steps of the proposed decryption algorithm.

DNA_DECODE(D, RM) :

p = pixel depth of image in bits
m, n = size of the image
Initialize an array X' of size $m \times n$
$count = 1$
for i = *1* **to** m **do**
  for j = *1* **to** n **do**
    foreach $p/2$ *nucleotides in* D **do**
      x = Binary values corresponding to D[$count$] as shown in Table 2 using Rule $RM[i][j]$
      $X'[i][j] = x$
      $count = count + p/2$
return X'

Algorithm 6. DNA decoding.

The process of decoding image is applied on the image received after applying substitution on every channel matrix using the rule map $RM4$, respectively as

$$\begin{aligned} \text{CR} &= \texttt{DNA_DECODE}(\texttt{sub}_\texttt{R}, RM4), \\ \text{CG} &= \texttt{DNA_DECODE}(\texttt{sub}_\texttt{G}, RM4), \\ \text{CB} &= \texttt{DNA_DECODE}(\texttt{sub}_\texttt{B}, RM4). \end{aligned} \tag{8}$$

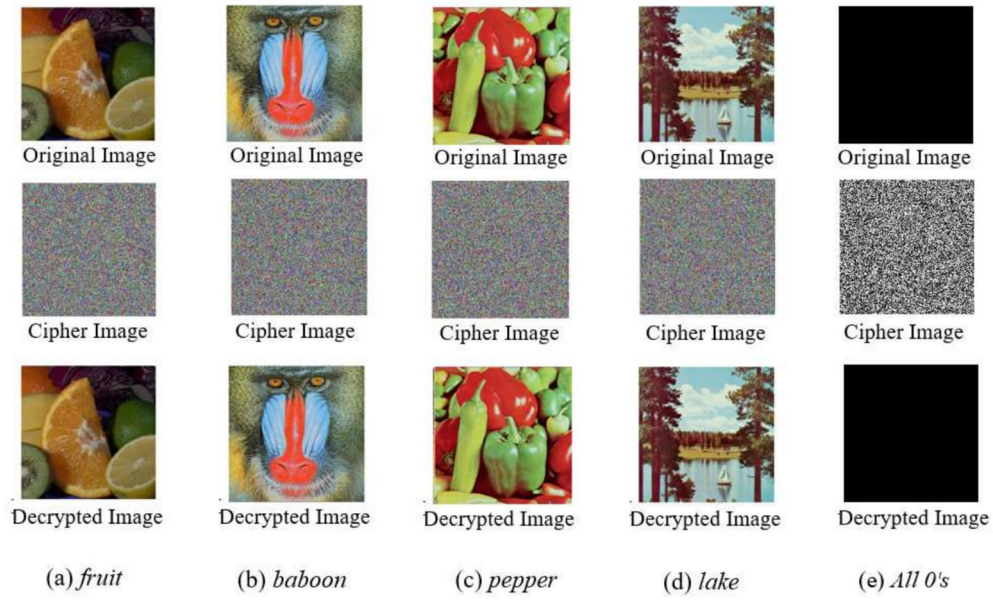


Fig. 10. Results of encryption and decryption by the proposed algorithm for the images—(**a**) fruit, (**b**) baboon, (**c**) pepper, (**d**) lake, and (**e**) All 0's. All the images have a dimension of 256×256.

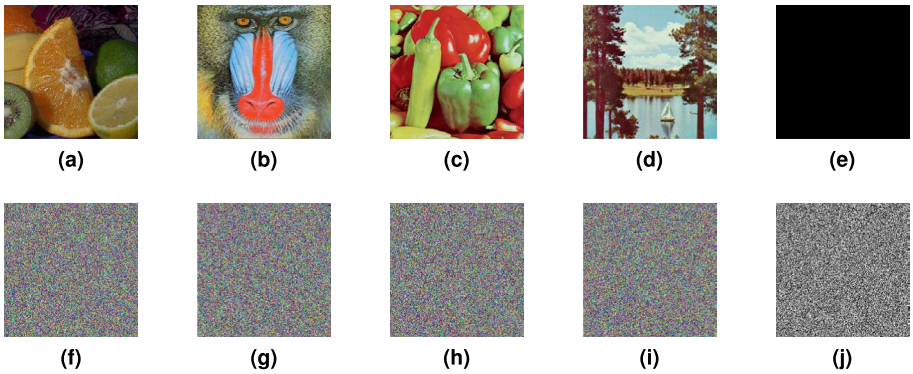


Fig. 11. Results of decryption of the images (**a–e**) using correct key, and (**f–j**) using key with one bit change from the original key.

Merging of the cipher channels

All the three decoded matrices obtained after Algorithm 6 are merged into final cipher image. The intermediate results of each step in the proposed encryption algorithm is shown in Fig. 7. The cipher image that is produced is noise-like and random, meeting the criteria for a secure image. This cipher picture is resistant to cryptographic attacks, has low correlation, and high entropy. This cipher image can be decrypted only with the knowledge of the correct encryption key. The decryption procedure is discussed in the next section.

Decryption algorithm

After obtaining cipher image we proceed for its decryption as on receiver side it needs to be decrypted. The encryption and decryption processes are inverted. The steps that are taken in last stages will now be taken in initial stages as we move in reverse fashion. As presented in flow chart, we again split the cipher images into R, G, B components and applies DNA encoding. If we compare encryption and decryption flow chart we can see we are dealing with DNAs in the later stages in encryption but in decryption we are using DNAs in initial stages. At first we do DNA encoding then we apply reverse DNA substitution using reverse substitution rules followed by DNA decoding. Encoding and Decoding have the same algorithm but substitution have minor change, it uses reverse substitution rules. After decoding we go for reverse spiral mixing which internally follows reverse fashion of mixing in encryption. Then we apply reverse modified ACM followed by reverse mix rows and reverse inter channel mixing. At last we again merge RGB into final decrypted image. The main point of decryption is to use the same key as it was used in encryption for getting 2D MCCM, initial vector, rules maps and DNA substitution map. All the steps in the decryption process are just reverse of the encryption ones. The proposed

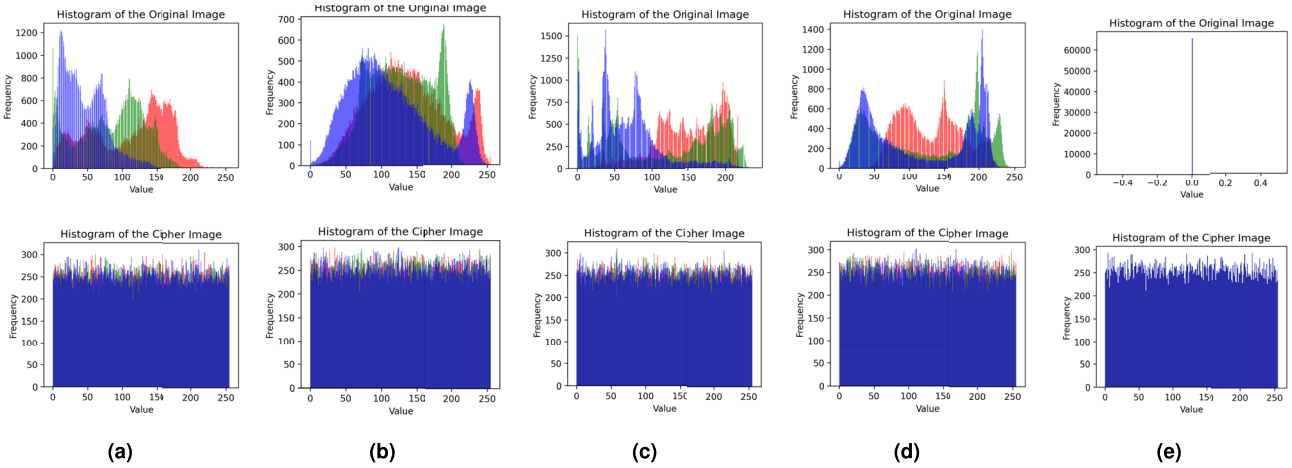


Fig. 12. Plain and cipher histogram of the five images shown in Fig. 10.

Image	Plain image			Cipher image		
	Red	Green	Blue	Red	Green	Blue
Fruit	3090.122	3090.257	3909.811	5453.239	5465.818	5475.349
Baboon	2889.991	1972.825	3383.311	5459.752	5444.902	5457.723
Pepper	1995.873	2556.325	1881.152	5469.795	5444.531	5485.011
Lake	1954.858	2572.911	2548.812	5469.953	5498.777	5421.363
All 0's	0	0	0	5440.601	5440.601	5440.601

Table 3. Variance analysis of the plain and cipher images.

Image	Plain image			Cipher image		
	Red	Green	Blue	Red	Green	Blue
Fruit	7.552	7.337	6.475	7.9971	7.9968	7.9974
Baboon	7.683	7.381	7.682	7.9972	7.9973	7.9971
Pepper	7.721	7.527	7.109	7.9972	7.9975	7.9972
Lake	7.331	7.638	7.341	7.9971	7.9974	7.9973
All 0's	0	0	0	7.9974	7.9974	7.9974

Table 4. Entropy of the plain and cipher images of dimension 256×256.

decryption process is shown in Figure 8, and the intermediate results after each step of decryption is shown in Fig. 9.

Experimental results and analysis

Numerous colour images were encrypted, and the cipher images' characteristics were examined in order to evaluate the cryptosystem's performance. The results of the encryption and decryption of some color images using the proposed cryptosystem has been shown in Fig. 10. The security analysis of the proposed cryptosystem has been presented in details in the following subsection.

Key space and key sensitivity

The encryption's resistance to brute-force attacks is tested by the length of a key. In aspect of cryptography, a high degree of security requires a key length that is greater than 2^{100}. In our proposed cryptosystem, The key used has a secured key space of 2^{240} and is 240 bits long. The key used in encryption and decryption is same because it is symmetric.

High sensitivity to the key is a fundamental requirement for the encryption system; otherwise, it will not be able to obtain the right decryption image, even in the event that there is a very slight shift in the input key. A small modification to the initial key will result in a unseen new decryption image for a perfect encryption scheme. In our experiment, we each alter four RGB colour images by merely varying the lone bit of the initial key as shown in Fig. 11. The decryption outcomes are displayed, along with the notable variations between the original (256×256) and decrypted images[18].

Histogram and variance analysis

A histogram describes how the image's pixel intensity values are distributed. The histograms of the plain and encrypted images are present in Fig. 12. It is evident that the cipher image's pixel values are consistently distributed throughout the interval [0, 255], in stark contrast to the plain image's pixel values which follows uneven trend. Single image shows the histogram for all three channels.

Additionally, we use histogram variances to statistically test the regularity of histograms. Mathematically, histogram variances are defined as

$$\mathrm{var}(I) = \frac{\sum_{x=1}^{m}\sum_{y=1}^{n}(I(x,y)-\mu)^2}{m \times n},$$

where $n \times n$ is the dimension of the image I, and μ is the mean of the image pixels' intensity, defined as

$$\mu = \frac{\sum_{x=1}^{m}\sum_{y=1}^{n} I(x,y)}{x \times y}.$$

Color images have low variance since they only have eight bits per pixel and have a non-uniform distribution of pixels in each R, G, and B component image. Table 3 displays natural images with variances less than 3000, while the matching cipher images of the corresponding Red, Green, and Blue components have variances greater than 5400.

Entropy analysis

The uncertainty measurement is provided by entropy. The entropy $\mathcal{E}(S)$ is calculated for a source S that contains n symbols, each with a probability of p_i as[19]

$$\mathcal{E}(S) = \sum_{i=1}^{n} p_i \log_2\left(\frac{1}{p_i}\right).$$

Entropy of plain images is low, but that of cipher images should be high and close to eight bits/symbol. Table 4 demonstrates that the encrypted images produced by the proposed method. The ideal value of entropy of cipher images of dimension 256×256 with three color channels should be greater than 7.900[20], and those generated by the proposed method have an entropy larger than 7.9950, indicating a high degree of randomness.

χ^2 analysis

The encrypted image's histogram offers a visual depiction of the data, while the χ^2 test offers a statistical analysis of the variation of pixel intensity readings. The χ^2 readings for an image of size $m \times n$ and pixel depth p bits can be computed as[21]

$$\chi^2 = \sum_{L=0}^{2^p-1} \frac{(x_L^0 - x_e)^2}{x_e},$$

where x_L^0 is the frequency that has been observed for the Lth intensity, where L is the gray level lying between 0 to $2^p - 1$, and x_e is the frequency expected, which is calculated as

$$x_e = \frac{2^p - 1}{m \times n}.$$

Table 5 shows the χ^2 values for the cipher images. The effective value of χ^2 for images with a pixel depth of 8 bits per pixel lies in the range of [210.7918, 293.2478][20], which is satisfied by the cipher images generated by the proposed algorithm.

Image	χ^2 of cipher image		
	Red	Green	Blue
Fruit	258.47	267.02	268.61
Baboon	232.32	268.43	267.44
Pepper	246.94	216.14	246.22
Lake	257.71	235.82	272.62
All 0's	253.21	253.21	253.21

Table 5. χ^2 of cipher images.

Image	Direction	Plain image			Cipher image		
		Red	Green	Blue	Red	Green	Blue
Fruit	Horizontal	0.9844	0.9879	0.9545	−0.0018	−0.0067	−0.0064
	Vertical	0.9856	0.9735	0.9857	0.0075	0.0018	−0.0077
	Diagonal	0.9433	0.9268	0.9402	0.0055	−0.0168	−0.0071
Baboon	Horizontal	0.9126	0.8962	0.9531	0.0031	0.0084	−0.0169
	Vertical	0.6595	0.7867	0.8541	0.0064	−0.0076	−0.0852
	Diagonal	0.7652	0.6806	0.8443	−0.0384	0.0239	0.0259
Pepper	Horizontal	0.9285	0.9465	0.9486	−0.0042	0.0092	−0.0086
	Vertical	0.9725	0.9772	0.9784	−0.0053	0.0041	0.0045
	Diagonal	0.9119	0.9455	0.8801	−0.0345	−0.0127	−0.0057
Lake	Horizontal	0.9322	0.9589	0.9647	0.0042	0.0082	0.0062
	Vertical	0.9794	0.9868	0.9863	−0.0085	−0.0063	0.0058
	Diagonal	0.8935	0.9496	0.9458	−0.0020	0.0074	−0.0069
All 0's	Horizontal	–	–	–	0.0039	0.0039	0.0039
	Vertical	–	–	–	0.0015	0.0015	0.0015
	Diagonal	–	–	–	0.0075	0.0075	0.0075

Table 6. Correlation of the plain and cipher images.

Image	NPCR, $\alpha = 0.001$			UACI, $\alpha = 0.001$		
	NPCR critical value = 99.5341			UACI range : [33.1593, 33.7676]		
	Red	Green	Blue	Red	Green	Blue
Fruit	99.6386	99.6118	99.5724	33.3419	33.4274	33.4146
Baboon	99.6062	99.6062	99.6154	33.3902	33.4853	33.2344
Pepper	99.5971	99.5986	99.6121	33.4408	33.4035	33.4181
Lake	99.5994	99.5986	99.6185	33.4693	33.3303	33.4835
All 0's	99.6017	99.6017	99.6017	33.4731	33.4731	33.4731

Table 7. NPCR and UACI.

Correlation analysis

Pixels have a strong correlation in plain images, however in crypted images, this correlation should be reduced. One among key statistical features of images is the correlation between neighbouring pixels, which indicates the limit of correlation between pixel values in adjacent positions in the image. The greater the efficiency of the developed encryption system, there is low correlation between the neighbouring pixels of the encrypted picture. Next, 10,000 pixels are chosen from both the encrypted and original images in order to examine the pixel correlation. The coefficient of correlation ρ is calculated as[22]

$$\rho(x,y) = \frac{\sum_{i=1}^{n}(x_i-\bar{x})(y_i-\bar{y})}{\sqrt{\sum_{i=1}^{n}(x_i-\bar{x})^2}\sqrt{\sum_{i=1}^{n}(y_i-\bar{y})^2}},$$

where n is the size of the two vectors with elements x_i and y_i having mean $\bar{x}$ and $\bar{y}$ respectively, and $-1 \leq \rho \leq 1$. The original images have a high positive correlation close to one, whereas the cipher images have no correlation with a value close to zero as shown in Table 6.

NPCR and UACI

Number of Pixel Changing Rate (NPCR) and *Unified Averaged Changing Rate* (UACI) are crucial metrics that assess a cryptosystem's resilience to differential attacks[23]. Let X and X' represent the cipher images that are generated following the encryption of a plain image of $m \times n$ size and a pixel depth of p, and by modifying one value of pixel of the initial image respectively. NPCR is the percentage of difference in the number of pixels between X and X', and is defined as[3]

$$\mathcal{N}(X,X') = \sum_{i=1}^{m}\sum_{j=1}^{n}\frac{\mathcal{D}(i,j)}{m\times n}\times 100\%,$$

where $\mathcal{D}$ is the number of pixels with changed values, calculated as

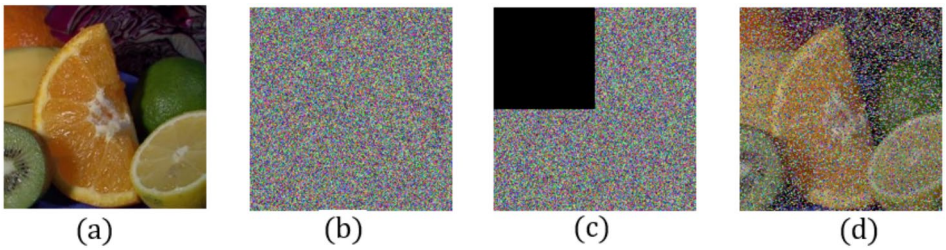


Fig. 13. Occlusion attack—(**a**) Plain image P, (**b**) Cipher image Z using the proposed encryption algorithm and a key $\mathcal{K}$, (**c**) 25% occlusion attack on Z to give Z', (**d**) Decrypted image D from Z' using the proposed decryption algorithm and the same key $\mathcal{K}$.

Image	25% occlusion attack		
	Red	Green	Blue
Fruit	8.23	8.25	8.21
Baboon	8.76	8.76	8.72
Pepper	8.84	8.76	8.76
Lake	8.81	8.81	8.82
All 0's	8.28	8.28	8.26

Table 8. PSNR on 25% occlusion attack on the cipher image.

Test	p-value	Result
Monobit	0.766	Passed
Frequency within block	0.721	Passed
Runs	0.34	Passed
Longest run ones in a block	0.338	Passed
Binary matrix rank	0.989	Passed
Discrete fourier transform	0.686	Passed
Non overlapping template matching	1	Passed
Maurers universal	0.999	Passed
Serial	0.147	Passed
Approximate entropy	0.027	Passed
Cumulative sums	0.049	Passed
Random excursion	0.957	Passed
Random excursion variant	0.695	Passed

Table 9. NIST randomness test on *Fruit* image of Fig. 10 of dimension 256×256 containing three color channels.

$$\mathcal{D}(i,j) = \begin{cases} 1, & \text{if } X(i,j) \neq X'(i,j) \\ 0, & \text{if } X(i,j) = X'(i,j). \end{cases}$$

UACI is the mean of the absolute difference between each pixel value of X and X' per bit, and is defined as[3]

$$\mathcal{U}(X,X') = \sum_{i=1}^{m}\sum_{j=1}^{n} \frac{|X(i,j) - X'(i,j)|}{(2^p - 1)\cdot(m \times n)}.$$

For an image of dimension 256×256 with a bit-depth of 8 bits per-pixel and a significance level $\alpha = 0.001$, the critical value of NPCR is 99.5341, and the range for UACI is from 33.1593 to 33.7676, the calculations of which are shown in Bhaya et al.[3]. Table 7 shows that the cipher image generated from the proposed algorithm has higher NPCR than the critical value, and its UACI lies well within the range. This shows that a single pixel change in the original image leads to a large change in the cipher image, protecting the system from differential attacks[24].

Image dimension	Inter-channel mixing	Mix rows	Rectangle ACM	Spiral mixing	DNA encoding	Substitution	DNA decoding	Total time
256×256	1.11	0.53	0.25	2.18	0.28	1.97	0.26	6.58
512×512	4.18	2.16	1.11	8.32	1.28	7.83	1.04	25.92
1024×1024	16.65	8.43	5.31	33.39	4.9	30.67	4.1	103.45

Table 10. Execution time (in seconds) of the various steps of the proposed encryption algorithm.

Image dimension	Image size (bytes)	Time (s)	Encryption throughput	Cycles (Per Byte)
256×256	196,608	6.58	29,879.64	107,096.35
512×512	786,402	25.92	30,340.74	105,468.75
1024×1024	3,145,728	103.45	30,408.24	105,234.78

Table 11. Computational analysis of the proposed encryption algorithm.

Noise and occlusion attack

During communication, a cipher image may be prone to noise and occlusion attack. However, knowing the correct key, considerable amount of the plain image should be recovered, so as to understand its visual content. *Peak Signal-to-Noise Ratio* (PSNR) is a useful metric for evaluating the effectiveness of a cryptosystem against noise attacks and occlusion. Let P be the plain image that is encrypted to create the cipher image Z using a key $\mathcal{K}$ and proposed encryption technique. The encrypted image is subjected to noise attack or occlusion to produce the image Z'. This image is then decrypted using the proposed decryption algorithm and the same key to provide the decrypted image D. The PSNR, for the image having dimension $m \times n$ with a bit-depth of p is defined as[3]

$$\text{PSNR} = 10 \times \log_{10} \left[\frac{(2^p - 1)^2}{\frac{1}{m \times n} \sum_{u=1}^{m} \sum_{v=1}^{n} \{P(u,v) - D(u,v)\}^2} \right].$$

An image needs a high PSNR—ideally more than 30 dB—to be visually discernible. However, because of their random noise like properties and high mean squared error in the associated pixels, cipher images have low PSNR values, typically less than 10 dB.

Figure 13 shows an example where a cipher image has been subjected to 25% occlusion attack and is decrypted using the same key. The decrypted image, though noisy, is good enough to be perceived visually. Table 8 shows the PSNR of the four test images when subjected to 25% occlusion attack.

NIST randomness test

The generated cipher images from the proposed algorithm have been tested using the NIST randomness test suite[25]. Table 9 shows the results of the test on the *Fruit* image in Fig. 10 of dimension 256×256 with three color channels, and their p-values. The test has been done using the `nistrng` package[26]. The values in the table, along with the pass results show the random behavior of the generated cipher and the robustness of the proposed algorithm against cryptography attacks.

Time complexity analysis

The proposed algorithm contains a number of steps which operate on the pixels og the images. To encrypt an image of dimension $m \times n$, an initial vector of dimension $1 \times m$ needs to be created, taking a time of $O(m)$. Each of the chaotic maps, generated thereafter, takes is of the same size as that of the image, therefore takes a time of $O(m \times n)$. Each of the operation of *Channel Splitting*, *Inter-Row Channel Splitting*, *Spiral Mixing*, *DNA Encoding*, *Substitution* and *DNA Decoding* operate on every pixel of the image once, therefore requiring a time of $O(m \times n)$. The rectangle ACM has a higher complexity, requiring a time of $O(\max(m,n) \times m \times n)$. Since all these operations run one after the other, the overall time complexity is the maximum asymptotic time taken by each of the steps, which turns out to be $O(\max(m,n) \times m \times n)$. Since the decryption algorithm has just the reverse steps of the encryption algorithm, the time complexity for the later also remains the same.

To evaluate the execution time, the *fruit* image shown in Fig. 10 of various resolution, particularly 256×256, 512×512, and 1024×1024 have been used. All the experiments have been run on a laptop having macOS, with Apple M1 processor having a 8-core CPU with a base frequency of 3.2 GHz and a RAM of 16 GB. The codes have been written in Python version 3.10. The time for various steps of the encryption algorithm is shown in Table 10.

To evaluate the execution time invariant of the dimension of image, the metric of *execution throughput* can be used[24] which is defined as

$$\text{Encryption Throughput} = \frac{\text{Image Size (bytes)}}{\text{Execution time (s)}}$$

Method	Encryption process	Entropy (expectation > 7.9)			NPCR ($\alpha = 0.001$) (critical value = 99.5341)			UACI ($\alpha = 0.001$) (range : [33.1593, 33.7676])		
		R	G	B	R	G	B	R	G	B
Original RGB color image		7.268	7.597	6.971	–	–	–	–	–	–
Proposed	Using $\otimes$	7.997	7.996	7.997	99.638	99.618	99.572	33.342	33.427	33.410
	Using XOR	7.997	7.996	7.996	51.416	51.416	51.428	18.390	18.380	18.380
	Using $(+, -)$	7.997	7.997	7.996	93.432	98.182	99.424	33.401	33.421	33.405
Ref[18]	Dynamic DNA encryption and chaos	7.997	7.996	7.997	99.600	99.600	99.610	33.560	33.460	33.450
Ref[9]	Fractional-order hyperchaotic system and DNA-coding	7.997	7.997	7.996	99.600	99.620	99.580	33.470	33.440	33.570
Ref[10]	Fractional fourier transform and DNA sequence operation	7.997	7.997	7.999	99.560	99.550	99.570	33.410	33.440	33.450
Ref[6]	Hybrid chaotic system and DNA sequences	7.997	7.997	7.997	99.640	99.600	99.610	33.470	33.360	33.440
Ref[27]	Hyperchaotic key transformed by singular value decomposition and RC5 encryption	7.999	7.999	7.999	99.580	99.600	99.600	29.580	28.040	30.690
Ref[28]	2D hyperchaotic sine couple map with inter and intra-block operations	7.997	7.997	7.997	99.630	99.630	99.630	33.490	33.460	33.490

Table 12. Comparative analysis of the proposed cryptosystem on *baboon* image of Fig. 10 of dimension 256×256.

. Since different machines have different configurations in terms of processor, speed and memory, a fair metric to evaluate the execution time can be presented in terms of *number of cycles per byte*[24], which is defined as

$$\text{Number of Cycles per Byte} = \frac{\text{CPU Speed (Hz)}}{\text{Encryption Throughput (bytes/s)}}$$

.

The computational analysis of the proposed encryption algorithm on various dimensions of images is shown in Table 11.

Comparative analysis

Using the same experimental setup and cryptosystem, the operator $\otimes$ has been compared with other operators such as XOR and $(+, -)$ on several color images. The NPCR and UACI tests demonstrate that the operator $\otimes$ outperforms XOR in terms of resistance to differential attacks, since the operator $\otimes$ yields satisfactory results whereas XOR fails horribly in these tests using with the suggested cryptosystem. While the $(+, -)$ operator yields fairly similar values for UACI and NPCR but also fails in some circumstances as well like for one channel fails and for other two channel give satisfactory result. However, the main disadvantage of $(+, -)$ operator is removing the pixel correlation and the suggested operator works significantly better in these situations. These findings support the assertion that the operator $\otimes$ outperforms the standard operators typically employed in traditional image cryptosystems. Table 12 provides an overview of various existing cryptosystem's performances and compare our values with theirs values on RGB color image. It can be seen that the proposed cryptosystem provides a robust security solution compared to some who have NPCR and UACI values out of the expected thresholds.

Conclusion and future scope

In our paper, we design a cryptosystem using a $\otimes$ operator for color images. The cryptosystem follows the properties of operator $\otimes$ which says it forms an Abelian group due to which it is used for encryption as well as decryption. This operation's application is shown in a suggested cryptosystem that includes DNA encoding, substitution, and decoding after a unique mixing operation that makes advantage of the distributive and commutative features of $\otimes$. The cryptosystem also demonstrate decryption process and algorithms where the changes are required for decryption. In contrast to the existing cryptosystems, which employ intricate encryption techniques or a number of rounds for confusion-diffusion, the proposed cryptosystem just requires basic computational operations. Analysis and experimental findings demonstrate that the suggested work satisfies the specifications needed for a safe images cryptosystem. Comparative analysis with other commonly used operators with color images shows that operator $\otimes$ outperforms them regarding color images encryption.

This paper proposes a cryptosystem based on the DNA operations, wherein the binary bits are converted to a sequence of DNA nucleotides using simple mappings of Watson-Crick complementary rules. Though this algorithm has been used in numerous DNA-based cryptosystems, the approach has been criticized owing to its fanciful nature and less relevance to DNA computing[29]. In fact, a number of codewords[30] and mappings[31] have been devised to generate sequences which follow multiple constraints to be used for DNA computing. Researchers can further study and explore algorithms to design cryptosystems which can operate on constrained DNA codes to be practically used in DNA computing models in the future.

Data availability

All data generated or analysed during this study are included in this published article [and its supplementary information files].

Received: 16 March 2025; Accepted: 23 May 2025
Published online: 30 September 2025

References

1. Mishra, P., Bhaya, C., Pal, A. K. & Singh, A. K. A medical image cryptosystem using bit-level diffusion with DNA coding. *J. Ambient. Intell. Humaniz. Comput.* **14**, 1731–1752 (2023).
2. Mishra, P., Bhaya, C., Pal, A. K. & Singh, A. K. A novel binary operator for designing medical and natural image cryptosystems. *Signal Process. Image Commun.* **98**, 116377 (2021).
3. Bhaya, C., Pal, A. K. & Islam, S. H. A novel image encryption and decryption scheme by using DNA computing. *Adv. Comput.* **129**, 129–172 (2023).
4. Cao, W., Mao, Y. & Zhou, Y. Designing a 2D infinite collapse map for image encryption. *Signal Process.* **171**, 107457 (2020).
5. Zhang, Q., Guo, L. & Wei, X. Image encryption using DNA addition combining with chaotic maps. *Math. Comput. Model.* **52**, 2028–2035 (2010).
6. Yaghouti Niyat, A. & Moattar, M. H. Color image encryption based on hybrid chaotic system and DNA sequences. *Multimedia Tools Appl.* **79**, 1497–1518 (2020).
7. Chai, X. et al. Color image compression and encryption scheme based on compressive sensing and double random encryption strategy. *Signal Process.* **176**, 107684 (2020).
8. Liu, Z., Wu, C., Wang, J. & Hu, Y. A color image encryption using dynamic DNA and 4-D memristive hyper-chaos. *IEEE Access* **7**, 78367–78378 (2019).
9. Dong, H., Bai, E., Jiang, X.-Q. & Wu, Y. Color image compression–encryption using fractional-order hyperchaotic system and DNA coding. *IEEE Access* **8**, 163524–163540 (2020).
10. Farah, M. B., Guesmi, R., Kachouri, A. & Samet, M. A novel chaos based optical image encryption using fractional Fourier transform and dna sequence operation. *Opt. Laser Technol.* **121**, 105777 (2020).
11. Benyahia, K., Khobzaoui, A. & Benbakreti, S. Hybrid image encryption: Leveraging DNA sequencing and Lorenz chaotic dynamics for enhanced security. *Clust. Comput.* **28**, 218 (2025).
12. Fetteha, M. A., Sayed, W. S. & Said, L. A. A lightweight image encryption scheme using DNA coding and chaos. *Electronics* **12**, 4895 (2023).
13. De Dieu, N. J., Ruben, F., Nestor, T., Zeric, N. T. & Jacques, K. Dynamic analysis of a novel chaotic system with no linear terms and use for DNA-based image encryption. *Multimedia Appl.* **81**, 10907–10934 (2022).
14. Alrubaie, A. H., Khodher, M. & Abdulameer, A. T. Image encryption based on 2dna encoding and chaotic 2d logistic map. *J. Eng. Appl. Sci.* **70**, 60 (2023).
15. Kheiri, H. & Dehghani, R. A hybrid model of recursive cellular automata, dna sequences, and chaotic system for image encryption. *Multimedia Tools Appl.* **1**, 1–27 (2024).
16. Yang, C., Wei, X. & Wang, C. S-box design based on 2d multiple collapse chaotic map and their application in image encryption. *Entropy* **23**, 1312 (2021).
17. Arnold, D. Math 45-linear algebra. *Sign* **1**, 3 (2000).
18. Chai, X., Fu, X., Gan, Z., Lu, Y. & Chen, Y. A color image cryptosystem based on dynamic DNA encryption and chaos. *Signal Process.* **155**, 44–62 (2019).
19. Chai, X. et al. An efficient approach for encrypting double color images into a visually meaningful cipher image using 2D compressive sensing. *Inf. Sci.* **556**, 305–340 (2021).
20. Liu, H., Liu, J. & Ma, C. Constructing dynamic strong S-box using 3D chaotic map and application to image encryption. *Multimedia Tools Appl.* **82**, 23899–23914 (2023).
21. Wang, X., Feng, L. & Zhao, H. Fast image encryption algorithm based on parallel computing system. *Inf. Sci.* **486**, 340–358 (2019).
22. Cohen, I. et al. Pearson correlation coefficient. In *Noise Reduction in Speech Processing* 1–4 (2009).
23. Wu, Y. et al. PCR and UACI randomness tests for image encryption. *J. Select. Areas Telecommun.* **1**, 31–38 (2011).
24. Qumsieh, R., Farajallah, M. & Hamamreh, R. Joint block and stream cipher based on a modified skew tent map. *Multimedia Tools Appl.* **78**, 33527–33547 (2019).
25. Rukhin, A. et al. *A Statistical Test Suite for Random and Pseudorandom Number Generators for Cryptographic Applications* Vol. 22 (US Department of Commerce, Technology Administration, 2001).
26. InsaneMonster. *GitHub—InsaneMonster/NistRng: Random Number Generator NIST Test Suite Framework for Python 3.6—SAILab—University of Siena*.
27. Alexan, W. et al. A new multiple image encryption algorithm using hyperchaotic systems, SVD, and modified RC5. *Sci. Rep.* **15**, 9775 (2025).
28. Chen, T., Huang, T., Zhang, Z., Lu, M. & Tang, J. Color image encryption based on 2D hyperchaotic Sine couple map and block scrambling. *IEEE Access* **1**, 1 (2024).
29. El Hanouti, I. & El Fadili, H. Security analysis of an audio data encryption scheme based on key chaining and DNA encoding. *Multimedia Tools Appl.* **80**, 12077–12099 (2021).
30. Benerjee, K. G. & Banerjee, A. On DNA codes with multiple constraints. *IEEE Commun. Lett.* **25**, 365–368 (2020).
31. Bhoi, S. S., Parampalli, U. & Singh, A. K. Construction of DNA codes with multiple constrained properties. *Cryptogr. Commun.* **16**, 1135–1149 (2024).

Author contributions

This paper is a part of the PhD work of C.B., who conceived the idea, experiments and proposed the algorithm. M.Z. contributed in the implementation of the code and enhancement of the algorithm. A.K.S. verified the methodology, code and the results. All the authors contributed in drafting and revising the overall manuscript.

Declarations

Competing interests

The authors declare no competing interests.

Additional information

Supplementary Information The online version contains supplementary material available at https://doi.org/10.1038/s41598-025-04021-4.

Correspondence and requests for materials should be addressed to A.K.S.

Reprints and permissions information is available at www.nature.com/reprints.

Publisher's note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Open Access This article is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License, which permits any non-commercial use, sharing, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if you modified the licensed material. You do not have permission under this licence to share adapted material derived from this article or parts of it. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit http://creativecommons.org/licenses/by-nc-nd/4.0/.

© The Author(s) 2025